



Universidad de Oviedo

Tema

Cadenas de Caracteres

Introducción al computador y el lenguaje C

Ingeniería de Sistemas y Automática - Universidad de Oviedo

Codificación de texto

Al igual que otros conceptos "abstractos" (días de la semana, tipos de fruta, etc.), el texto se codifica como números enteros:

- Cada carácter de un texto está codificado por un valor entero.
- Todos los computadores deben utilizar la misma codificación para entenderse entre ellos: códigos ASCII y UNICODE.
- Puesto que no hay muchos caracteres diferentes, no es necesario un elevado nº de bits para codificarlos todos:
 - ASCII: 8 bits = 256 caracteres diferentes.
 - UNICODE: 16 bits = 65536 caracteres diferentes.

Se utiliza el tipo de datos `char` para indicar un entero con el nº de bits necesarios (8/16) para codificar un carácter.

Para declarar constantes se pone el carácter entre comillas simples, para evitar recordar el código de cada carácter.

**Ejs: 'a' ≡ 97 'b' ≡ 98 'c' ≡ 99 '.' ≡ 46 '0' ≡ 48
'1' ≡ 49 '\n' ≡ 10**

Codificación de texto

```
char letra;  
letra = 'a';
```

Lo que realmente existe

01100001

Es un entero que ocupa 1 byte en memoria por ser char.

Como entero (97)

A cada bit le corresponde un peso:
 $1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^0 = 97$

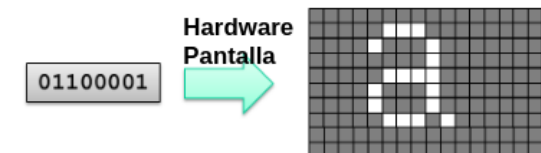
Las op. aritméticas (+, -, *, /) dan lugar a valores de la misma forma.

```
suma = letra + 4; //  
103
```

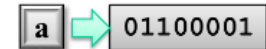
Como carácter ('a')

En pantalla, esta secuencia de bits genera los puntitos de la 'a'.

En un teclado: la tecla envía al computador el código binario 01100001.



En un teclado es aún más evidente: ¡¡¡ la 'a' es una pegatina !!! Simplemente, cuando se pulsa sobre la tecla que tiene esa pegatina, el hardware de teclado envía al computador el código binario



La tabla de códigos ASCII

ASCII utiliza 8 bits = 256 valores diferentes:

Estándar (0 a 127)

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	\$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

Extendidos (128 a 255)

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	À	160	A0	á	192	C0	Ĺ	224	E0	α
129	81	Á	161	A1	â	193	C1	Ľ	225	E1	β
130	82	Â	162	A2	ã	194	C2	Ţ	226	E2	Γ
131	83	Ã	163	A3	ä	195	C3	ţ	227	E3	π
132	84	Ä	164	A4	å	196	C4	ı	228	E4	Σ
133	85	Å	165	A5	Ä	197	C5	†	229	E5	σ
134	86	Å	166	A6	Å	198	C6	‡	230	E6	μ
135	87	Ç	167	A7	°	199	C7	‡	231	E7	τ
136	88	Ç	168	A8	¿	200	C8	Ĳ	232	E8	φ
137	89	È	169	A9	ˆ	201	C9	ƒ	233	E9	Θ
138	8A	È	170	AA	˜	202	CA	Δ	234	EA	Ω
139	8B	É	171	AB	˜	203	CB	ƒ	235	EB	σ
140	8C	Ê	172	AC	˜	204	CC	ƒ	236	EC	∞
141	8D	Ë	173	AD	ı	205	CD	=	237	ED	ω
142	8E	Ë	174	AE	«	206	CE	ƒ	238	EE	τ
143	8F	Ë	175	AF	»	207	CF	Δ	239	EF	Π
144	90	Ê	176	B0	˜	208	D0	Δ	240	F0	±
145	91	Ê	177	B1	˜	209	D1	ƒ	241	F1	±
146	92	Ë	178	B2	˜	210	D2	ƒ	242	F2	≥
147	93	Ë	179	B3		211	D3	Δ	243	F3	≤
148	94	Ë	180	B4		212	D4	Δ	244	F4	ƒ
149	95	Ë	181	B5		213	D5	ƒ	245	F5	ƒ
150	96	Ë	182	B6		214	D6	ƒ	246	F6	÷
151	97	Ë	183	B7	ƒ	215	D7	ƒ	247	F7	≈
152	98	Ë	184	B8	ƒ	216	D8	ƒ	248	F8	*
153	99	Ë	185	B9	ƒ	217	D9	ƒ	249	F9	*
154	9A	Ë	186	BA	ƒ	218	DA	ƒ	250	FA	·
155	9B	Ë	187	BB	ƒ	219	DB	ƒ	251	FB	·
156	9C	Ë	188	BC	ƒ	220	DC	ƒ	252	FC	·
157	9D	Ë	189	BD	ƒ	221	DD	ƒ	253	FD	·
158	9E	Ë	190	BE	ƒ	222	DE	ƒ	254	FE	■
159	9F	Ë	191	BF	ƒ	223	DF	ƒ	255	FF	□

Operaciones con caracteres

Se opera igual que con datos enteros (son enteros).

```
char c1, c2;
c1 = 'a';
c2 = c1 + 2; // c1=97 (0x61), c2=99 (0x63)='c'

if (c2 >= 'a' && c2 <= 'z')
    printf("El carácter %c es una letra minúscula\n", c2);
```

Funciones específicas de interés:

- `char getchar();` // Pide un carácter por consola
- `putchar(char c);` // Escribe un carácter en consola
- `int isalpha(char c);` // Devuelve V si c está entre 'a'..'z' ó 'A'..'Z'
- `char toupper(char c);` // Devuelve mayúscula si era minúscula

¡Ojo con caracteres extendidos! Ñ, ñ, á, é, etc. no siguen reglas de ordenación estándar.

Códigos para caracteres "especiales" comienzan con \: '\n', '\0', '\"', '\\'.

Cadenas de caracteres (string)

Un texto es una secuencia ordenada de caracteres, por tanto en C será un array de char (en inglés *string*).

Gestión clásica (Inadecuada para strings)

Requiere dirección de comienzo y longitud.

```
char txt[4];  
int n = 4;  
txt[0] = 'H';  
txt[1] = 'o';  
txt[2] = 'l';  
txt[3] = 'a';
```

Gestión en C (Carácter Nulo)

Se gestiona colocando un código especial (0) al final de la tabla.

```
char txt[5]; // Un elemento más  
txt[0] = 'H';  
txt[1] = 'o';  
txt[2] = 'l';  
txt[3] = 'a';  
txt[4] = 0; // o '\0'
```

El string queda definido **únicamente por su dirección de comienzo**. El final se produce donde se encuentre el carácter de código 0 ($0 = '\0' \neq '0' = 48$).

Cadenas de caracteres (string)

Se pueden manejar constantes de tipo string, colocando el texto entre comillas dobles:

```
char txt[5] = "Hola";
```

Índice	Valor	Dirección
txt[0]	'H'	3000 = txt
txt[1]	'o'	3001
txt[2]	'l'	3002
txt[3]	'a'	3003
txt[4]	0	3004
txt[5]	?	3005

Una constante tipo string es un `const char*`, y se puede utilizar en cualquier lugar del código que lo acepte:

```
// En <stdio.h>
int printf(const char* fmt, ...);

// En nuestro programa
int x = 3, y = 2;
printf("x vale %d, y vale %d\n", x, y); // Recibe dirección de comienzo
```

Cadenas de caracteres (string)

Sólo se puede asignar el contenido de un string a una constante en el momento de su declaración:

```
char txt[5] = "Hola"; // Correcto
char nom[5];
nom = "Pepe";         // Incorrecto, ¡equivale a 3100 = 4000!
nom = txt;             // Incorrecto, ¡equivale a 3100 = 3000!
```

txt[0]	'H'	3000 = txt	nom[0]	?	3100 = nom	'P'	4000 = "Pepe"
txt[1]	'o'	3001	nom[1]	?	3101	'e'	4001
txt[2]	'l'	3002	nom[2]	?	3102	'p'	4002
txt[3]	'a'	3003	nom[3]	?	3103	'e'	4003
txt[4]	0	3004	nom[4]	?	3104	0	4004
txt[5]	?	3005	nom[5]	?	3105	?	4005
...	...	...	...	...	...	...	...

Cadenas de caracteres (string)

Todas las operaciones con string requieren recorrer los elementos de la tabla uno por uno, terminando al alcanzar el carácter 0:

```
char txt[5] = "Pepe";
char nom[5];
int i, iguales;

// Copiar txt a nom
for (i = 0; txt[i] != 0; i++) {
    nom[i] = txt[i];
}
nom[i] = 0; // ¡Añadir terminador nulo! (nom vale lo mismo que txt)

// Comparar nom y txt
if (nom == txt) // MAL: Siempre falso (3000 == 3100)

iguales = 1;
for (i = 0; txt[i] != 0; i++) {
    iguales = iguales && (txt[i] == nom[i]);
}
// Aquí: iguales (V o F) indica si son iguales
```

txt[0]	'P'	3000 = txt	nom[0]	?	3100 = nom
txt[1]	'e'	3001	nom[1]	?	3101
txt[2]	'p'	3002	nom[2]	?	3102
txt[3]	'e'	3003	nom[3]	?	3103
txt[4]	0	3004	nom[4]	?	3104
txt[5]	?	3005	nom[5]	?	3105
...	...	...	...	...	...

nom[0]	'P'	3100 = nom
nom[1]	'e'	3101
nom[2]	'p'	3102
nom[3]	'e'	3103
nom[4]	0	3104
nom[5]	?	3105
...	...	...

Operaciones con strings

Funciones para E/S de strings:

- `char* gets(char* cad);` // Pide una cadena por consola
- `int puts(const char* cad);` // Escribe una cadena en consola

Se utiliza `%s` en `printf()` y `scanf()`.

Ejemplo:

```
char nombre[20];
char apellido[20];

printf("Nombre: ");
gets(nombre);
printf("Apellido: ");
gets(apellido);
printf("Nombre completo: %s %s\n", nombre, apellido);
```

Funciones de cadena de caracteres

Realizan los bucles más típicos en el tratamiento de strings (copiar, comparar, añadir, buscar, ...). Trabajan con tablas de char terminadas en el caracter 0. Incluir `<string.h>`.

- Trabajan con tablas de char terminadas en el caracter 0.
- Incluir `<string.h>`.

Funciones más utilizadas:

<code>int strlen(const char* cad);</code>	Devuelve longitud de cadena
<code>char* strcpy(char* dst, const char* src);</code>	Copia src en dst
<code>char* strncpy(char* dst, const char* src, int n);</code>	Copia máx n caracteres (no incl. nulo)
<code>char* strcat(char* dst, const char* src);</code>	Añade src al final de dst
<code>char* strncat(char* dst, const char* src, int n);</code>	Añade máx n caracteres (incl. nulo)

Nota: El valor devuelto suele ser el mismo que dst en strcpy/strcat.

Funciones de cadena de caracteres

...Funciones más utilizadas

<code>int strcmp(const char* str1, const char* str2);</code>	Compara códigos ASCII. Devuelve: 0 $\Rightarrow$ iguales, <0 $\Rightarrow$ str1 anterior, >0 $\Rightarrow$ str1 posterior.
<code>int strncmp(const char* str1, const char* src, int n);</code>	Id. con máximo n caracteres.
<code>char* strchr(const char* str, char c);</code>	Busca caracter c en cadena cad.
<code>char* strstr(const char* str, const char* substr);</code>	Busca subcadena substr en cadena str.

Funciones de conversión de datos

Convierten cadenas de caracteres de/hacia tipos de datos numéricos.

Funciones más utilizadas:

<code>int atoi(const char* cad);</code>	Obtiene entero decimal
<code>double atof(const char* cad);</code>	Obtiene valor real
<code>double strtod(const char* cad, char** ptEnd);</code>	Obtiene valor real y puntero al final de conversión
<code>int sscanf(const char* cad, const char* fmt, ...);</code>	Id. a <code>scanf()</code> pero obteniendo datos de cadena
<code>char* itoa(int num);</code>	Obtiene cadena decimal para entero
<code>char* ftoa(double num);</code>	Obtiene cadena decimal para real
<code>int sprintf(char* cad, const char* fmt, ...);</code>	Id. a <code>printf()</code> pero almacenando en cadena

Funciones de cadena de caracteres

Ejemplo: pedir nombre y apellido, unirlos, y comprobar si se trata de "Marie Curie".

```
char nombre[20];
char apellido[20];
char* todo;

printf("Nombre: ");
gets(nombre);
printf("Apellido: ");
gets(apellido);

// Reserva espacio dinámico (+1 para espacio, +1 para \0)
todo = (char*) malloc(strlen(nombre) + 1 + strlen(apellido) + 1);

strcpy(todo, nombre);
strcat(todo, " ");
strcat(todo, apellido);

if (strcmp(todo, "Marie Curie") == 0)
    puts("Es una científica\n");

free(todo);
```

Funciones de cadena de caracteres

Ejemplo: pedir comando de texto y extraer valor entero en una variable (.... POS = 90).

```
char texto[40];
int posicion;
char* cmd;

printf("Comando: ");
gets(texto);

cmd = strstr(texto, "POS"); // Busca subcadena
if (cmd != NULL)
{
    // Avanza puntero saltando 'POS' y espacios en blanco
    for (cmd = cmd + strlen("POS"); *cmd == ' '; cmd++)
        ;

    if (*cmd == '=')
        posicion = atoi(cmd + 1); // Extrae número tras '='
}
```

Conversión y Formato de Cadenas

Funciones de conversión numérica y formateo de texto (<stdlib.h> y <stdio.h>)

1. Texto a Número (atoi, atof, strtod)

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    // 1. atoi: Texto a entero (int)
    int entero = atoi("452"); // 452

    // 2. atof: Texto a real (double)
    double real = atof("3.14159"); // 3.14159

    // 3. strtod: Texto a double + puntero al final
    char entrada[] = "12.50 Voltios";
    char *fin;
    double v = strtod(entrada, &fin);
    // v = 12.50 | fin -> " Voltios"

    return 0;
}
```

Nota de uso: Útiles para procesar comandos recibidos por puerto serie o ficheros donde los datos numéricos vienen formateados como texto.

2. Formateo y Extracción (sscanf, sprintf)

```
#include <stdio.h>

int main(void) {
    // 1. sscanf: Extrae datos desde un buffer
    char reg[] = "Sensor_01 23.5";
    char nombre[20];
    float temp;
    sscanf(reg, "%s %f", nombre, &temp);

    // 2. sprintf: Construye una cadena formateada
    char mensaje[50];
    int id = 101;
    sprintf(mensaje, "ID: %d -> Temp: %.1f C", id, temp);
    // Reemplaza de forma estándar a itoa() / ftoa()

    return 0;
}
```

Cadenas de caracteres

A tener en cuenta al usar cadenas de caracteres:

- Son tablas como cualquier otra: requieren reserva de espacio en memoria, no hay comprobación de salida de rango, etc.
- No se usa la longitud de la tabla, sino el código especial 0. ¡No se puede olvidar añadir el carácter 0 al manipular strings!
- Se pueden manipular:
 - Carácter por carácter, usando índices o punteros.
 - Usando funciones de la librería estándar.
 - Creando funciones propias, sin parámetro de longitud.

Siempre hay que asegurar que el tamaño reservado es capaz de albergar los datos:
`char txt[5] = "Pepe"; strcat(txt, " Pérez"); // Incorrecto`
Es habitual el uso de asignación dinámica de memoria.

Ejercicios propuestos

1. Pedir por consola el nombre de una unidad lógica, directorio y archivo, y componer el camino completo hacia el archivo (ej: "c:\user\practicas\ejemplo.txt").
2. Comprobar si el archivo anterior tiene extensión "txt" y se encuentra en el subdirectorio "practicas".
3. Realizar una función que, dado un comando con el formato "POSICION=valor entero", devuelva el valor entero. Admitir que la palabra clave POSICION sea POS, POSI, POSIC o POSICION, y que haya espacios en blanco antes y después del '='.
4. Añadir a la función anterior la posibilidad de devolver un código de error si no se encuentra el comando o el formato es incorrecto.
5. Pedir por consola una cadena con el formato "[v1,v2,v3,...vn]" y extraer de la misma los contenidos de una tabla de float con esos valores.